

Universidad del Valle de Guatemala

CC2008 - Introduccion a la Programacion Orientada a Objetos
Semestre II, 2026

Laboratorio 2

Agencia de detectives - Caso misterioso

Analisis y diseno

Nombre: _____

Carne: _____

1. Requisitos funcionales

1. Solicitar al iniciar el nombre, codigo y detective responsable del caso.
2. Crear un caso nuevo que reemplace al actual y comience sin ubicaciones ni pistas.
3. Guardar como maximo cinco ubicaciones en un arreglo basico de objetos; los espacios libres contienen null.
4. Guardar para cada ubicacion codigo, nombre, direccion o descripcion, riesgo y estado.
5. Validar que el riesgo se encuentre entre 1 y 10; un valor invalido genera `IllegalArgumentException`.
6. Registrar una ubicacion en una posicion valida y libre del arreglo, con datos validos.
7. Consultar todas las ubicaciones sin mostrar los espacios null.
8. Consultar una ubicacion por posicion e informar si la posicion es invalida o esta vacia.
9. Modificar el riesgo y estado de una ubicacion existente.
10. Descartar una ubicacion asignando nuevamente null a su posicion.
11. Guardar una cantidad dinamica de pistas directamente en un `ArrayList<Pista>`.
12. Guardar para cada pista codigo, descripcion, tipo, importancia y confiabilidad.
13. Validar importancia entre 1 y 10 y confiabilidad entre 0 y 100; los valores invalidos generan `IllegalArgumentException`.
14. Registrar pistas sin permitir codigos repetidos.
15. Consultar todas las pistas e informar cuando la coleccion este vacia.
16. Buscar una pista por codigo recorriendo el `ArrayList`.
17. Modificar codigo, descripcion, tipo, importancia y confiabilidad de una pista existente, sin permitir que el nuevo codigo quede repetido.
18. Eliminar del `ArrayList` una pista localizada por codigo.
19. Reportar ubicaciones registradas, espacios disponibles y ubicacion de mayor riesgo.
20. Reportar cantidad de pistas, pista de mayor importancia, pista de mayor confiabilidad y promedio de importancia.
21. Evitar calculos que requieran elementos cuando el `ArrayList` este vacio.
22. Manejar entradas numericas incorrectas mediante `InputMismatchException`, limpiar la entrada y continuar.
23. Usar bloques try-catch y al menos un finally con una accion final pertinente.
24. Presentar un menu con las trece operaciones solicitadas y mantenerlo activo hasta seleccionar Salir.

2. Clases identificadas

Clase	Proposito
Ubicacion	Representa un lugar investigado, conserva sus datos y valida sus cambios de riesgo y estado.
Pista	Representa una evidencia encontrada, conserva sus datos y valida importancia y confiabilidad.
Caso	Representa la investigacion y administra el arreglo fijo de ubicaciones y el <code>ArrayList</code> dinamico de pistas, incluidas las busquedas y estadisticas.
Main	Es el driver program: solicita entradas, muestra el menu, captura excepciones y coordina las operaciones sobre el caso.

3. Atributos

Clase	Atributo y tipo	Proposito
Ubicacion	-codigo: String	Identifica la ubicacion.
	-nombre: String	Nombre del lugar.
	-direccion: String	Direccion o descripcion del lugar.
	-nivelRiesgo: int	Riesgo entre 1 y 10.
	-estado: String	Situacion actual de la ubicacion.
Pista	-codigo: String	Identificador unico de la pista.
	-descripcion: String	Detalle de la evidencia.
	-tipoEvidencia: String	Clasificacion de la evidencia.
	-nivelImportancia: int	Importancia entre 1 y 10.
	-nivelConfiabilidad: int	Confiabilidad entre 0 y 100.
Caso	-MAX_UBICACIONES: int = 5	Define la capacidad fija.
	-nombre: String	Nombre de la investigacion.
	-codigo: String	Identificador del caso.
	-detectiveResponsable: String	Responsable de la investigacion.
	-ubicaciones: Ubicacion[5]	Arreglo basico fijo; sus posiciones libres contienen null.
	-pistas: ArrayList<Pista>	Coleccion dinamica de pistas.
Main	-SCANNER: Scanner	Lee las entradas del usuario durante toda la ejecucion.

4. Metodos

Clase	Metodo	Funcion
Ubicacion	+Ubicacion(String, String, String, int, String)	Inicializa y valida todos los datos.
	+getNivelRiesgo(): int	Devuelve el riesgo para el reporte.
	+actualizar(int, String): void	Valida y cambia riesgo y estado.
	-validarRiesgo(int): void	Genera excepcion si el riesgo no esta entre 1 y 10.
	+toString(): String	Devuelve la informacion para mostrarla.
Pista	+Pista(String, String, String, int, int)	Inicializa y valida todos los datos.
	+getCodigo(): String	Permite buscar y detectar duplicados.
	+getNivelImportancia(): int	Proporciona la importancia para los calculos.
	+getNivelConfiabilidad(): int	Proporciona la confiabilidad para los calculos.

	+actualizar(String, String, String, int, int): void	Valida y modifica todos los datos.
	+toString(): String	Devuelve la informacion para mostrarla.
Caso	+Caso(String, String, String)	Crea un caso con estructuras vacias.
	+registrarUbicacion(int, Ubicacion): void	Valida posicion y disponibilidad antes de guardar.
	+obtenerUbicacion(int): Ubicacion	Obtiene una posicion valida y no nula.
	+modificarUbicacion(int, int, String): void	Actualiza una ubicacion existente.
	+descartarUbicacion(int): void	Deja la posicion en null.
	+mostrarUbicaciones(): void	Recorre el arreglo y omite posiciones nulas.
	+registrarPista(Pista): void	Rechaza duplicados y agrega al ArrayList.
	+buscarPista(String): Pista	Recorre el ArrayList y devuelve la coincidencia o null.
	+modificarPista(...): void	Localiza y actualiza una pista.
	+eliminarPista(String): void	Localiza y elimina una pista.
	+cantidadUbicaciones(), espaciosDisponibles()	Calculan ocupacion del arreglo.
	+ubicacionMayorRiesgo(), pistaMayorImportancia(), pistaMayorConfiabilidad()	Recorren las estructuras para obtener los maximos.
	+cantidadPistas(), promedioImportancia()	Calculan cantidad y promedio de pistas.
Main	+main(String[]): void y metodos privados de solicitud, menu y reporte	Controlan el ciclo del menu, entradas, salidas y manejo de excepciones.

5. Respuestas al analisis

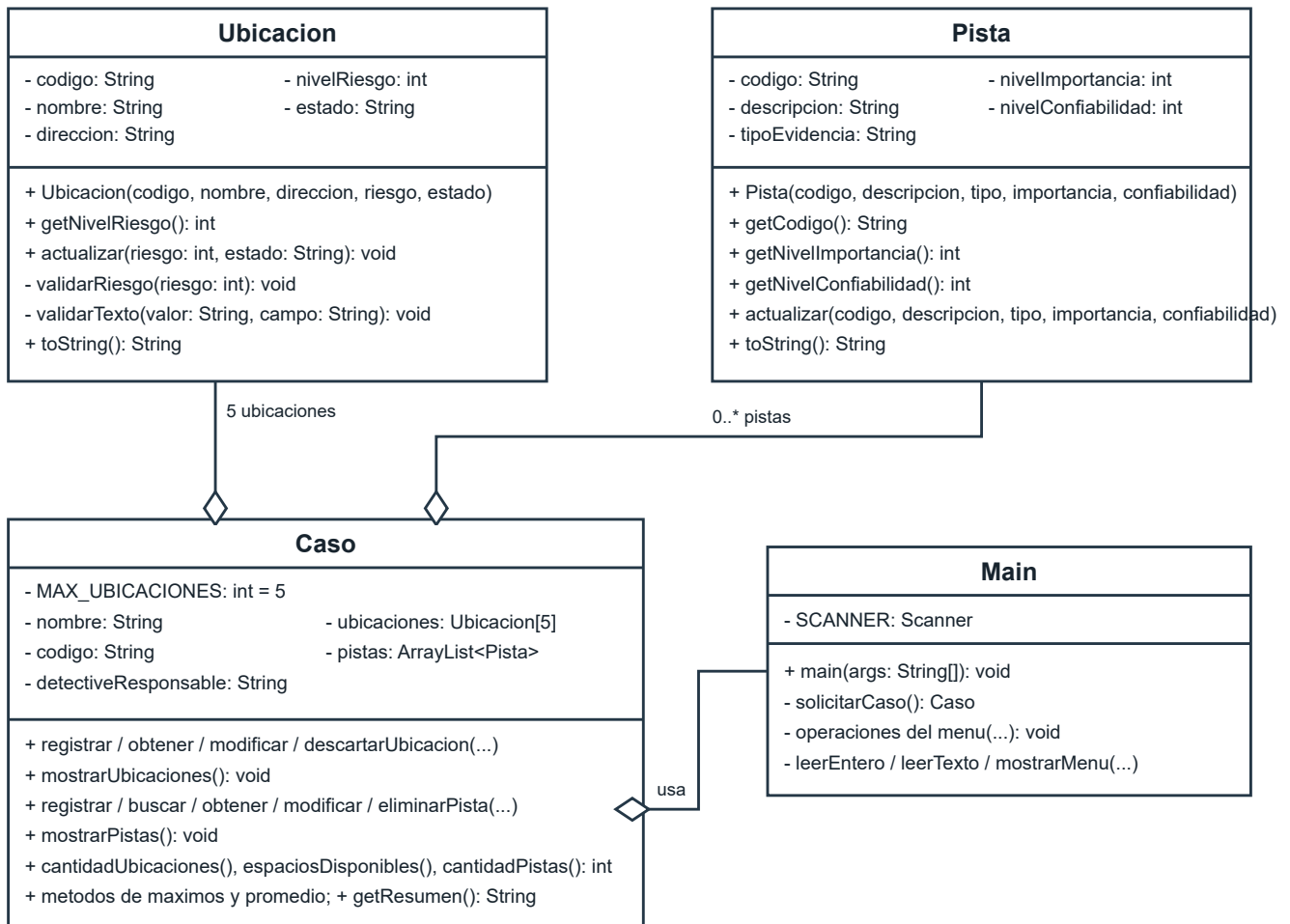
1. **Propiedades y metodos:** se detallan en las secciones 3 y 4. Las entidades conservan y validan sus datos; Caso administra ambas colecciones; Main controla la interfaz.
2. **Tipos:** los textos son String, niveles y posiciones son int, el promedio es double, las operaciones sin resultado son void y las busquedas devuelven el objeto correspondiente.
3. **Arreglo basico:** ubicaciones es Ubicacion[] de tamaño 5 porque el limite se conoce desde el inicio.
4. **ArrayList:** pistas es directamente ArrayList<Pista> porque la cantidad de evidencias es desconocida.
5. **Visibilidad:** los atributos son privados para conservar el encapsulamiento. Las operaciones usadas desde otras clases son publicas y las validaciones y ayudas internas son privadas.
6. **Parametros:** los constructores reciben los datos iniciales; registrar recibe posicion y objeto; buscar o eliminar recibe codigo; modificar recibe el identificador actual y todos los nuevos valores.
7. **Inicializacion y validacion:** los constructores asignan los datos, crean el arreglo y el ArrayList. Antes de cambiar estado se validan textos no vacios, riesgo e importancia de 1 a 10 y confiabilidad de 0 a 100.
8. **Posiciones nulas:** se compara ubicaciones[i] != null antes de invocar metodos. Una consulta sobre null genera una excepcion controlada con un mensaje.

9. **Operaciones en ArrayList:** la busqueda recorre con un ciclo `for-each` y compara codigos sin distinguir mayusculas. Modificar usa el objeto encontrado y eliminar usa `remove`.
10. **Excepciones:** `InputMismatchException` aparece si `Scanner` recibe texto donde se espera un entero; `IllegalArgumentException` aparece por rangos, textos, posiciones u operaciones invalidas. `Main` las captura para conservar el menu. El `finally` de la lectura numerica limpia siempre el resto de la linea.

6. Manejo de excepciones

Situacion	Excepcion / control	Respuesta
Texto en una entrada numerica	<code>InputMismatchException</code>	Se informa el error; <code>finally</code> limpia la linea y el menu continua.
Riesgo, importancia o confiabilidad fuera de rango	<code>IllegalArgumentException</code>	Se muestra el mensaje y no se almacena el objeto invalido.
Posicion fuera de 0 a 4, ocupada o vacia	<code>IllegalArgumentException</code>	Se evita acceder o modificar incorrectamente el arreglo.
Codigo repetido o pista inexistente	<code>IllegalArgumentException</code>	Se evita duplicar, modificar o eliminar un elemento inexistente.
Colecciones vacias	Validacion previa	Se muestran mensajes apropiados y se evitan calculos sin elementos.

7. Diagrama de clases UML



Anotacion: Caso compone exactamente cinco espacios de Ubicacion y una cantidad variable de objetos Pista. Main depende de Caso para ejecutar las operaciones del menu.